SkillChain — Blockchain-Based Skill Passport

Decentralized Academic & Skill Credential Verification Project

DOMAIN: Blockchain & Web3 Technologies | NETWORK: Ethereum (Solidity, Ethers.js, React)

1. Introduction & Background

In the contemporary academic and professional sphere, verifiable credentials—including university degrees, diplomas, and technical course certificates—serve as vital proof of competence for recruitment and higher education. However, traditional paper certificates and standard digital PDFs are highly vulnerable to unauthorized alteration, duplication, and forgery using basic editing software. Centralized institutional verification systems frequently suffer from high administrative overhead, long response latencies, single points of failure, and security vulnerabilities.

SkillChain resolves these challenges by introducing an immutable, decentralized skill passport powered by Ethereum smart contracts and client-side cryptographic SHA-256 hashing. Certifying organizations anchor cryptographic proofs of issued credentials directly on the blockchain, enabling instantaneous, tamper-evident verification by recruiters, universities, and students without relying on intermediaries.

2. Problem Statement

- **Credential Forgery:** Digital PDFs lack cryptographic integrity; modifying student names, skills, or grades requires minimal technical skill.
- **Centralized Vulnerability:** University databases are targets for cyberattacks, internal database tampering, and server outages.
- **Verification Latency:** Background verification currently relies on manual inquiries, taking days or weeks to authenticate credentials.
- **Unverifiable Revocation:** If a credential is cancelled due to academic malpractice, verifiers holding downloaded copies have no direct way to detect that the certificate has been invalidated.

3. Project Objectives

1. Develop an Ethereum Solidity smart contract (SkillChain.sol) to store cryptographic certificate hashes permanently on-chain.
2. Implement deterministic client-side SHA-256 hashing to guarantee that any modification in metadata invalidates verification immediately.
3. Enforce unique certificate identifiers to eliminate duplicate or conflicting registrations on the blockchain ledger.
4. Implement decentralized access control restricting certificate revocation strictly to the authorized issuing authority.
5. Provide a responsive, beginner-friendly React.js frontend interface for issuing, viewing, and verifying certificates in real time.
6. Ensure reliability through a comprehensive automated unit test suite covering registration, verification, and revocation edge cases.

4. System Architecture & Workflow

The architecture consists of three integrated layers: a client frontend (React + Vite), client-side cryptographic hashing, and a decentralized smart contract on an Ethereum local network (Hardhat).

Workflow Stage	Operational Mechanism
1. Issuance	Issuer inputs student name, skill title, organization, and issue date. The app assigns a unique ID (SKC-YYYY-XXXX), computes the canonical SHA-256 hash, and broadcasts it to SkillChain.sol.
2. Anchoring	The smart contract verifies the ID is unused, logs the transaction with an event, records the issuer address and timestamp, and stores the hash immutably.

3. Verification	A verifier enters the Certificate ID. The application calculates the SHA-256 digest and queries the smart contract. The contract performs an on-chain comparison and returns **VERIFIED**, **INVALID**, or **REVOKED**.

5. Key Modules

Module	Functional Description
Landing Page	Professional dashboard highlighting the project mission (*'Your Skills. Your Proof.'*) with direct access to issuance, viewing, and verification portals.
Add Certificate	Captures student and course details, validates inputs, computes SHA-256 digests, and executes Ethereum smart contract registration with transaction receipt feedback.
My Certificates	Displays all user-issued certificates stored in local browser storage, allowing live on-chain status checks and authorized revocation.
Verification Portal	Public verification page comparing candidate-provided details against immutable on-chain records with explicit status badges.
SkillChain Contract	Solidity smart contract maintaining an internal mapping of certificate records, enforcing duplicate rejection, and restricting revocation to the original issuer.

6. Technology Stack

- **Frontend:** React.js 18, Vite 5, JavaScript (ES6+), Responsive Custom CSS3, Lucide React Icons
- **Smart Contract:** Solidity ^0.8.20
- **Blockchain Environment:** Hardhat Local Blockchain Network (Chain ID: 31337)
- **Web3 Integration:** Ethers.js (v6) for JSON-RPC provider and signer interactions
- **Client Storage:** Browser localStorage for offline credential persistence
- **Testing & Tooling:** Hardhat Test Runner, Mocha, Chai assertion library

7. Security & Cryptographic Mechanisms

Data Privacy & Fingerprinting via SHA-256: Sensitive personal details (student identity, grades) are hashed prior to registration. Because SHA-256 is a one-way mathematical function, no private student information is exposed on the public blockchain, yet any minor alteration in certificate attributes generates a completely different hash, instantly invalidating forged documents.

- **Immutability:** Once committed to an Ethereum block, certificate records cannot be edited, deleted, or backdated.
- **Issuer Access Control:** The revokeCertificate method enforces require(cert.issuer == msg.sender), barring unauthorized addresses from revoking credentials.
- **Duplicate Collision Prevention:** The contract ensures require(!certificates[_certificateId].exists) to eliminate duplicate registrations.

8. Automated Testing & Validation Results

A rigorous test suite containing **10 automated tests** was executed via Hardhat and Chai with a **100% pass rate**:

- ✔ Registering a certificate on-chain with unique ID and SHA-256 hash
- ✔ Rejecting registration with empty ID or hash
- ✔ Rejecting tampered / mismatched hashes
- ✔ Authorizing revocation by the legitimate issuer
- ✔ Rejecting verification of revoked certificates
- ✔ Rejecting duplicate certificate IDs
- ✔ Verifying authentic certificate hashes
- ✔ Handling non-existent certificate queries
- ✔ Rejecting revocation attempts by unauthorized accounts
- ✔ Preventing duplicate revocation calls